TA Guide — Lab 5: pavo Introduction

Quantifying reef-fish color patterns (Week 6)

EEB 187 — Ecology & Evolution of Color

May 6, 2026

Table of contents

Purpose of this guide

This document is for **Rosamari** (and any future TA stepping into Lab 5). It pairs with three other things:

Artifact	Where	Purpose
`scripts/walkthrough.R`	repo root	the live-coded R script you and the class will run together
`lab-05-pavo-intro.qmd`	repo root	the student worksheet with Part A (demo) + Part B (their three fish)
`pre-lab-installation.qmd`	repo root	what students were asked to install before lab

This guide goes **deeper than the slide deck** on the conceptual basis for everything in `walkthrough.R`, then walks through the script step by step with **(a) what it computes, (b) why it matters biologically, (c) student questions to expect, and (d) troubleshooting for the errors you will actually see in lab.**

Read this *before* the first time you teach the lab. Skim it again the morning of, especially the troubleshooting boxes. Total reading time: ~45 minutes.

Part 1 — What the lab teaches

The pedagogical claim

> Color *patterns* — vertical bars, horizontal stripes, reticulate spots — can be reduced to a small set of numbers that distinguish them automatically. The single most important number is the **A statistic** (aspect ratio).

By the end of lab the students should be able to:

1. Predict the rough A value of a fish from looking at its photo (`A > 1`, `A < 1`, `A  1`).

2. Run the pipeline themselves on a new fish image (segment → classify → adjacent → metrics).
3. Explain to a peer *why* `A` distinguishes vertical from horizontal patterns — at the level of "we count adjacent pixel pairs and direction matters."
4. Critique a result. When they get unexpected metrics, they should be able to point to *which step* (segmentation, k choice, white-background exclusion) is the likely culprit.

What's NOT in scope today

These come later in the term:

- Spectral analysis from spectrometers (`procspec`, `peakshape`)
- Visual modeling through animal eyes (`vismodel`, `coldist`, JNDs) — Lab 6
- Many-species comparative analysis with phylogeny — Lab 9
- Endler's saturation/luminance space arguments at the *receiver* level — Lecture 14–16

If a student asks "but what does the *fish that's looking at this fish* see?" — that's the right question, and the answer is "we'll get there in Lab 6 with `vismodel()`." For today, every metric is **photographic**, not perceptual.

Part 2 — Conceptual primer

This is the layer below the slide deck — the math and the *why*. Students don't need all of this. You do.

2.1 The Endler 2012 framework

The whole pipeline is built on John Endler's 2012 *Biol. J. Linn. Soc.* paper "A framework for analysing colour pattern geometry: adjacent colours." The core move:

1. **Discretize** the image into a small number of color classes (k-means in RGB).
2. **Sample** the classified image on a regular grid.
3. **Count** transitions between color classes between adjacent grid cells — separately for horizontal vs. vertical adjacency.
4. **Summarize** the transition matrix with a small set of geometry-aware statistics.

Step 4 produces the metrics students will see:

Metric	What it measures	Intuition
k	number of color classes	how many "colors" the fish has
m	transition density (off-diagonal mass / total)	how *busy* the pattern is per unit area
A	vertical edges / horizontal edges (the **aspect ratio**)	direction — bars vs. stripes vs. isotropic

Metric	What it measures	Intuition
Jc	color heterogeneity (Simpson on color-class areas)	how *evenly* the colors are used
Jt	transition heterogeneity (Simpson on transitions)	how diverse the *boundaries* are
m_dS	mean chromatic boundary strength	how *saturated* a typical edge is
m_dL	mean luminance boundary strength	how *bright/dark* a typical edge is

The A statistic is the headline because:

- It cleanly distinguishes **bars (A $>$ 1)**, **stripes (A $<$ 1)**, and **isotropic patterns (A 1)** in three lines of R.
- It's invariant to color choice — a yellow-and-black bar code and a red-and-white bar code give the same A.
- It's a *geometric* claim about pattern, not a perceptual one — so it's a clean entry point before we add receiver biology in Lab 6.

2.2 Why segmentation matters (Step 1.5 of walkthrough)

The full pipeline is `image → classify → adjacent → metrics`. Every step downstream of `classify()` is poisoned by background pixels:

- **k-means** allocates color classes to *whatever pixels exist*. If half the photo is blue water, one of your k classes will be water, and you'll have one less class for the fish.
- **Adjacent grid sampling** drops a regular xpts $\times$ ypts grid over the *whole image*. Long uniform stretches of background drown out the fish's directional signal and pull A toward 1.

Three demo fish on uncropped photos will give A 1 across the board — the lab's punchline disappears. **The dramatic A values (1.54 / 0.60 / 1.02) are entirely a product of segmentation.** This is why the demo fish are pre-segmented and why Step 1.5 has three Paths.

> **i** The white-background trick
>
> We segment to a white background and then *include* white as a +1 in the k argument: `kcols = (fish_colors + 1)`. The white class becomes one of the k centroids, and `bkgID + exclude = "background"` tells `adjacent()` to ignore transitions involving white. Net effect: same as analyzing only the fish, but in one R call without a polygon mask.

2.3 What `classify()` actually does

`classify(img, kcols = k)` runs **k-means on each pixel's RGB triplet**:

1. Treats each pixel as a point in 3D color space (R, G, B).
2. Initializes k centroids randomly.

3. Iteratively assigns each pixel to its nearest centroid, then moves each centroid to the centroid of its assigned pixels. Repeats until stable.
4. Returns an integer-valued array (the "classified image"), plus a `classRGB` attribute giving the k centroid colors.

This is a *purely color-based* clustering — it knows nothing about spatial structure. A 3-pixel red dot in the corner gets the same class as a 3-pixel red dot on the body. Spatial structure enters at the *next* step.

> ⚠ k-means is non-deterministic without a seed
>
> k-means initializes centroids randomly. Two runs on the same image with the same k can produce slightly different palettes. pavo's `classify()` uses a fixed seed under the hood for reproducibility, but if students compare each other's runs, the *order* of color classes (class 1, 2, 3, …) may differ even when the colors themselves are the same.
> **Practical implication:** when comparing two students' fish, compare RGB centroids in `classRGB`, not class indices.

2.4 What `adjacent()` actually does

`adjacent()` is the heart of the pipeline. It does five things in one call:

1. **Drops a grid** of `xpts` × `xpts` sample points over the classified image. xpts = 200 means a 200 × 200 grid.
2. **Visits each grid cell** and reads its color class.
3. **Builds a transition matrix T**: for each pair of horizontally-adjacent grid cells, increment `T[c1, c2]`. Then for each vertically-adjacent pair, increment `T[c1, c2]` again — but kept as a *separate* horizontal-T and vertical-T.
4. **Excludes background** (if `bkgID + exclude` set): drops the row + column for the white-bg class.
5. **Computes the seven Endler metrics** from the (filtered) horizontal-T and vertical-T.

The A statistic is just `sum(off_diag(vertical_T)) / sum(off_diag(horizontal_T))`:

- A vertical-bar fish has lots of yellow-to-black horizontal transitions but very few vertical transitions within a bar. Vertical edges (between bars) » horizontal edges within bars. **A > 1.**
- A horizontal-stripe fish: opposite. **A < 1.**
- A reticulate / spotty fish: roughly equal in both directions. **A 1.**

> ℹ Why xpts = 200, not 50, not 1000?
>
> - **xpts = 50** → too sparse; the grid skips small features (e.g., the cleaner wrasse stripe is only ~20% of the body, so at 50 pts you sample it at maybe 10 cells, with high variance).
> - **xpts = 1000** → expensive (400× more work) and over-samples noise; small JPEG artifacts start showing up as transitions.
> - **xpts = 200** → empirically robust on Endler's example datasets and our test fish. This is the default in his 2012 paper.

2.5 Why `coldists` matters for `m_dS` and `m_dL`

Without a `coldists` table, `adjacent()` returns NA for the boundary-strength metrics. The reason: `m_dS` (mean chromatic boundary strength) is the *average chromatic distance across edges, weighted by edge frequency*. To compute that you need to know the chromatic distance between every pair of color classes — that's `coldists`.

In `walkthrough.R` we compute a *photographic* `coldists` from RGB (Euclidean distance in RGB space). This is a stand-in for *perceptual* coldists, which would come from `vismodel() %>% coldist()` once we have a fish-eye visual model in Lab 6.

So: the m_dS and m_dL students will see today are "geometric saturation steps in RGB," not "JNDs as a fish would see them." Be ready for the question; flag the upgrade in Lab 6.

2.6 The three demo fish — what to expect

Demo fish	Family	Pattern	Predicted A	Measured A (typical)
Sergeant major	Pomacentridae	Vertical bars	A $>$ 1	~**1.5**
Cleaner wrasse	Labridae	Horizontal stripe	A $<$ 1	~**0.6**
Mandarin fish	Callionymidae	Reticulate	A 1	~**1.0**

Tiny variation between runs (±0.05) is normal — k-means seeds and slight differences in xpts grid alignment.

Part 3 — Step-by-step walkthrough of `scripts/walkthrough.R`

This is the section to keep open while you teach. Each subsection is one numbered step in the script.

Step 0 — Setup

```
library(pavo)
library(magick)
library(tidyverse)
source("scripts/helpers.R")     # ← critical: loads simple_coldists,
                                #   pick_white_bg, pick_col, segment_fish,
                                #   composite_to_white, flip_to_left_lateral
packageVersion("pavo")
```

What's happening: loading the three packages plus checking we're on pavo 2.9. The `magick` package is the image-IO backend pavo uses internally; `tidyverse` is for `bind_rows`, `tibble`, `pivot_longer`, `ggplot`. The `source("scripts/helpers.R")` call brings in every function used downstream — without it, the first call to `simple_coldists()` errors with "could not find function".

Why this matters: if any of these errors out, nothing downstream works. Get students to "all four loaded clean" before proceeding. The `source()` line is the one students forget when they jump into Part B from a fresh session — emphasize it.

⚠ Troubleshooting Step 0

Symptom	Cause	Fix
`there is no package called 'pavo'`	install never ran or hit an error	`install.packages("pavo")` and watch for non-zero exit status; check `R --version` 4.3
`unable to load shared object … magick.so`	system ImageMagick missing	macOS: `brew install imagemagick`; Linux: `sudo apt install libmagick++-dev`. Then *reinstall* the R `magick` package so it relinks
`package 'pavo' was built under R version 4.X` warning	pavo binary built on newer R than running	warning only, ignore — works fine

Step 1 — Load the three demo fish (segmented)

```
sergeant_major <- getimg("images/demo/segmented/sergeant-major.jpg")
cleaner_wrasse <- getimg("images/demo/segmented/cleaner-wrasse.jpg")
mandarin_fish  <- getimg("images/demo/segmented/mandarin-fish.jpg")
```

What's happening: `getimg()` reads a JPG into pavo's `rimg` class — internally a 3D array (height × width × {R, G, B}) with a few attributes (`imgname`, the original filename).

Why three fish, why these three: they were curated specifically to span the A axis cleanly. The sergeant major has near-perfect verticality (bars top-to-bottom of the body); the cleaner wrasse has a dominant horizontal black stripe with weak vertical structure; the mandarin's labyrinthine pattern is roughly isotropic in horizontal vs. vertical edges. If a student asks why we don't pick X — the answer is "the demo is calibrated for clean teaching; in Part B you'll see messier real-world A values."

Plot the three side-by-side:

```r
par(mfrow = c(1, 3))
plot(sergeant_major); title("sergeant major")
plot(cleaner_wrasse); title("cleaner wrasse")
plot(mandarin_fish);  title("mandarin fish")
par(mfrow = c(1, 1))
```

Pause here. **Have students predict A** before computing anything. This is the pedagogical hook — they need to commit to a prediction before seeing the answer.

> Calling on students at this point
>
> Pick any student and ask: "Which one will have the highest A? Why?" If they say "the bars one because vertical edges," you have a class that's tracking. If they hedge, push: "What does A measure?" Get them to surface the definition before you reveal numbers.

> Troubleshooting Step 1

Symptom	Cause	Fix
`cannot open the connection` / `cannot open file`	working directory is wrong	`getwd()` to see where R thinks it is; `setwd()` to the lab folder, or use **Session → Set Working Directory → To Source File Location**
Image loads but `plot()` shows it black or distorted	rare — corrupted JPG	re-clone the repo; check the file is non-empty (`file.size("images/demo/segmented/sergeant-major.jpg")`)
`Error: object 'getimg' not found`	`library(pavo)` skipped	back to Step 0
`Error: could not find function "simple_coldists"` (or `pick_white_bg`, etc.)	`source("scripts/helpers.R")` skipped	back to Step 0; this is the most common Step 3 error in our test runs

Step 1.5 — Segmentation (the conceptual punchline)

This is *the* step to spend time on. Most lab questions trace back to a misunderstanding here.

The visual demo: open three images side-by-side from `images/demo/`:

- `images/demo/raw/sergeant-major.jpg` — what came off Wikipedia
- `images/demo/sergeant-major.jpg` (if present — cropped to bounding box)
- `images/demo/segmented/sergeant-major.jpg` — what we use

Walk through visually: each step strips more of "the photograph" away from "the fish." The metrics get cleaner as the photo gets cleaner.

Conceptual explanation to give students (use your own words; this is the gist):

> If we ran classify() on the raw photo, k-means would happily allocate one or two of our k classes to the blue water and the rocks. We'd lose those classes for the actual fish patterning. And when adjacent() drops its grid, half of the cells would land in featureless background and just dilute the directional signal. **Segmentation isn't a cosmetic clean-up — it's the difference between measuring the fish and measuring the photograph.**

Three paths students can use (worksheet has all three; we use Path 1 for the demo):

1. `segment_fish()` in R — wraps rembg + ImageMagick. Best quality.
2. **Web tool + `composite_to_white()`** — remove.bg → transparent PNG → magick composites onto white in R.
3. **Manual crop** in Preview.app — fastest fallback, slightly weaker metrics.

⚠ Troubleshooting Step 1.5

Symptom	Cause	Fix
`segment_fish()` errors with "rembg failed"	`uvx` and `rembg` neither on PATH	confirm `uvx --help` works in terminal; if yes, in R run `Sys.getenv("PATH")` — if `~/.local/bin` missing, restart RStudio; still failing, fall back to Path 2 (web tool)
`magick: not authorized` (Linux)	Debian's ImageMagick policy.xml blocks JPG by default	the `.binder/Dockerfile` patches this; for native Linux, `sudo nano /etc/ImageMagick-6/policy.xml` and change `rights="none"` to `rights="read|write"` for JPEG and PNG
Output JPG looks like the fish has been "hollowed out"	rembg's u2net model failed on this fish (rare for fish but happens for unusual angles)	fall back to Path 3 (manual crop); note the rembg model is trained on humans + objects, not fish-specific
Web tool returns transparent PNG, but `getimg()` says "image is empty"	student forgot `composite_to_white()` and tried to load PNG directly	`composite_to_white()` first, then `getimg()` on the resulting `*-segmented.jpg`

Step 2 — Classify each image

```
sm_class <- classify(sergeant_major, kcols = 4)
cw_class <- classify(cleaner_wrasse, kcols = 4)
mf_class <- classify(mandarin_fish,  kcols = 6)
```

What's happening: k-means on RGB. Each pixel snapped to one of k color centroids. Returns an `rimg` of integers (1 to k) with a `classRGB` attribute.

Why these particular k values:

- **Sergeant major (k=4):** yellow body + black bar + pale belly + white background = 4
- **Cleaner wrasse (k=4):** dark stripe + mid-blue body + light belly + white bg = 4
- **Mandarin (k=6):** orange + deep blue + mid blue + dark + accent + white bg = 6 (more colors → more classes)

The +1 rule: segment-to-white means we add 1 to k relative to the number of fish colors, to give the white background its own class.

Live-coding tip — show classify on the wrong k

Run `classify(sergeant_major, kcols = 2)` and plot it. Students will see the yellow-and-black bars become **one** class with white as the other — the pattern is destroyed. Then run with k=4 and show how the bar structure pops back. This makes the "k matters" claim visceral.

Troubleshooting Step 2

Symptom	Cause	Fix
Classified image looks washed out / 2 colors when k=4	one class collapsed because its mean was very close to another	re-run; k-means random init can occasionally land on a degenerate minimum
`classRGB` shows two near-identical rows (e.g., two pale grays)	k is too high for the actual color diversity	reduce k by 1
`Error: 'kcols' missing`	typo, e.g., `kcol = 4`	spelled `kcols` (with the s)

Inspect the palettes: the white-bg class is "the one with R G B 1." We use this fact in Step 3 to find and exclude it automatically with `pick_white_bg()`.

Step 3 — Adjacency (the heart of the lab)

This is the step that produces the numbers. Students don't need to follow every line — they need to follow the *concept*.

```r
sm_cd <- simple_coldists(sm_class)        # pairwise RGB distances
sm_bkg <- pick_white_bg(sm_class)         # which class is white
sm_adj <- adjacent(sm_class, xpts = 200, xscale = 5,
                   coldists = sm_cd,
                   bkgID = sm_bkg, exclude = "background")
```

What's happening, in three sentences:

1. `simple_coldists()` computes pairwise RGB Euclidean distances between every pair of color classes. Tells `adjacent()` what counts as a "big" boundary vs. a "small" one.
2. `pick_white_bg()` finds the brightest class (highest R+G+B sum) and returns its index. That's the white-background class.
3. `adjacent()` drops a 200×200 grid, counts horizontal and vertical color-class transitions, *excludes* the white-bg class from the count, and computes the seven metrics.

The xscale = 5 argument: ostensibly the body length in cm. *Only matters if you care about distance metrics in real-world units.* For this lab we care about ratios (A) and density (m), so xscale could be 1 or 100 with no effect. Tell students "leave it at 5; doesn't change the lab."

Why exclude "background": without this, the long uniform stretches of white pad the transition counts with within-white-bg pairs (which are zero transitions, but the area inflates the denominator of m). A drops; m drops; the contrast between the three fish gets muddier.

If a student asks 'why not just crop the white off and skip exclude?'

Cropping to the fish's bounding box leaves white CORNERS still in the image (any non-rectangular fish silhouette has them). The exclude approach handles arbitrary-shaped fish silhouettes correctly. It's also faster than re-cropping every fish.

Troubleshooting Step 3

Symptom	Cause	Fix
`m_dS` and `m_dL` return NA	`coldists` not passed	check `simple_coldists(class_img)` ran and was assigned to `*_cd`
All metrics look weird, A way off	`bkgID` set to the *wrong* class index	check `pick_white_bg(class_img)` actually points to white. Compare `attr(class_img, "classRGB")[bkg_idx, ]` — should be ~(1, 1, 1)
`Error: bkgID must be in 1:kcols`	bkgID computed for one fish was used on another	re-compute `*_bkg` from each fish's own classified image

`xpts = 200` runs slowly (~5 sec per fish)	normal on a laptop	mention to class — bigger grids = slower. 200 is the speed/accuracy sweet spot
Result has weird column names	older pavo version uses `Asp` instead of `A`	`pick_col()` already handles this; students see the metric name from the tibble printout

Step 4 — Extract metrics

The `pick_col()` helper grabs the seven metrics across all three fish into a single tibble. **Nothing computational happens here — it's just data wrangling.** Students don't need to follow every line; they need to read the resulting tibble and confirm A goes 1.5 / 0.6 / 1.0.

Show the print-out and pause

```
# A tibble: 3 × 9
  fish           family       k     m       A    Jc    Jt   m_dS   m_dL
  <chr>          <chr>     <dbl> <dbl>   <dbl> <dbl> <dbl>  <dbl>  <dbl>
1 sergeant_major Pomacent…     4 0.42   1.54   0.71  …      …      …
2 cleaner_wrasse Labridae      4 0.31   0.60   0.62  …      …      …
3 mandarin_fish  Callionym…    6 0.55   1.02   0.81  …      …      …
```

Stop and let this sink in. Ask: "Did the prediction hold?" Get a verbal yes from the class before moving on to the visualization.

Step 5 — Predict vs. measure visualization

```
metrics %>%
  pivot_longer(c(m, A, Jc, m_dL), names_to = "metric", values_to = "value") %>%
  ggplot(aes(x = fish, y = value, fill = family)) + …
```

What's happening: four metric values × three fish = a 4-panel grouped bar plot. Visual confirmation of what the table already showed.

Why this matters pedagogically: numbers tell, plots sell. The bar chart with A on its own panel is the moment students see the pattern claim made visible. The three families each get a distinct color → easy comparison.

⚠ Troubleshooting Step 5

Symptom	Cause	Fix
Plot is empty / no bars	`pivot_longer` operated on wrong column names	check `colnames(metrics)` — m, A, Jc, m_dL must all be present
Plot displays but axes are weird	NA values in some metrics (e.g., m_dS missing)	the plot will skip NAs — note in narration if it happens
`Error: object 'pivot_longer' not found`	`library(tidyverse)` skipped	back to Step 0

Step 6 — "Now you try" — student practice

This is where students take over and you become the help desk. **The Step 6 template in `walkthrough.R` now points at a real pre-loaded path** (`images/student-fish/chaetodontidae/chaetodon-auriga.jpg`) so students can run the template as-is to verify their pipeline works, then swap in their assigned-lineage species.

The most common student errors during Step 6 are:

⚠ Top 5 student questions during Step 6 (in order of frequency)

1. **"My A came back as 0.4 but I thought it would be 1.2!"** — Almost always either a wrong k value or wrong bkgID. Walk over, ask `attr(my_class, "classRGB")` — does class 1 look like white? If not, `pick_white_bg(my_class)` to find which class IS white.
2. **"`segment_fish()` is hanging"** — rembg's u2net model is being downloaded for the first time (~150 MB). Tell them to wait 60 seconds. If it still hangs, fall back to Path 2 (web tool).
3. **"My fish image is too small / pixelated"** — the xpts=200 grid is too dense for tiny images. Either find a larger source image, or the metrics are unreliable (note this in worksheet).
4. **"I picked three vertical-bar fish — what now?"** — they need diverse pattern types. Send them back to FishView / their lineage folder to find one of each pattern.
5. **"`getimg()` says cannot open file"** — file path. `getwd()` in R; `setwd()` to lab folder.

Part 4 — Pedagogical playbook

4.1 Time budget for the 90-minute lab

Block	Time	What you're doing
Demo Part A — walkthrough.R Steps 0–5	35 min	live-code with the three demo fish; pause at each prediction point
Transition + Q	5 min	"Now your turn" — read worksheet Part B Step 1 with the class
Independent Part B	35 min	students run pipeline on their three fish; you triage
Wrap-up + cross-lineage discussion	10 min	pick 2–3 groups to share; compare their A values to the demo
Submit reminder	5 min	due-by-EOD reminder, point to BruinLearn

If you run long on demo, drop the `m_dS` / `m_dL` discussion (they can read it in the worksheet). Don't drop A — it's the headline.

4.2 What to emphasize in Part A

The two ideas the lab pivots on:

1. **Segmentation is structural, not cosmetic.** Dwell on Step 1.5. Students who skip this skim of the lab end up with confused metrics in Part B and they don't understand why.
2. **A is a directional signal.** Spend two minutes on the *intuition* — "vertical edges divided by horizontal edges, weighted by where they appear" — before showing the computed A values.

Everything else is bookkeeping (k choice, white-bg exclusion, simple_coldists).

4.3 What to emphasize in Part B

In their independent practice, push students to:

1. **Predict before computing.** Have them write down their predicted A for each of their three fish *in the worksheet* before running adjacent(). Then have them update it after measurement.
2. **Reason about disagreements.** When a fish's measured A doesn't match their prediction, that's the most pedagogically valuable moment of the lab. Don't let them just write "the A came out at 0.9, weird." Push for "I predicted $A > 1$ because I saw bars, but the bars are partial / oblique / contiguous with the dorsal stripe, which adds horizontal edges I didn't count."
3. **Compare across the room.** In the wrap-up, compare three students from three different lineages. Did the A statistic recover the same pattern dimensions across families that haven't seen each other in 100M years? (Spoiler: yes, because A is geometric, not phylogenetic.)

4.4 Where students get stuck (and what to do)

Stuck on	What's actually wrong	Your move
"It's not running"	almost always a typo or missing `library(pavo)`	run their session's `search()` to confirm pavo is loaded
"k-means feels arbitrary"	they're right — it is, partially	tell them so. The lab teaches reproducible measurement, not an oracle. Different k gives different metrics; they pick the k that matches the visible color count, then defend their choice
"What does the metric *mean*?"	conflating measurement with interpretation	A *measures* edge-direction ratio. *Interpretation* — "does this fish look striped to a predator?" — needs Lab 6's `vismodel()`. Be explicit about the layering
"My fish doesn't have an A pattern type"	they picked three reticulate fish from the same family	normal for some lineages (e.g., Acanthuridae has lots of disruptive coloration). Tell them: pick the three that show the most contrast in pattern within their lineage; the A values may all cluster, and that's a finding worth noting

---

Part 5 — Worksheet rubric (suggested)

The worksheet has three assessable sections. Suggested scoring out of 10:

Component	Points	What you're looking for
Three completed metric tables (Part B Step 3)	3	tables filled, all 7 metrics present, k chosen reasonably for each fish
Comparison answers — A prediction vs measurement (Part B Step 4a)	2	honest yes/partially/no answer with a *reason*; "no, because" is fine if reasoned
Cross-lineage comparison (Part B Step 4b)	2	identifies a similar demo fish, names *which metrics* are similar/different, not just "they both look striped"

Component	Points	What you're looking for
Biological interpretation (Part B Step 5)	3	proposes a plausible selection pressure linked to the *measured* metrics; doesn't have to be correct, but must be reasoned. "I'm uncertain — here's why" answers full marks if the reasoning is solid

Common reasons to deduct:

- Tables half-empty with no explanation
- "A < 1, A > 1, A 1, just like the prediction" with no interpretation when the values disagreed
- Selection hypothesis that doesn't reference any of their measured metrics ("disruptive coloration" with no link to high m or high m_dS)
- Wholesale reliance on the demo fish without any engagement with their own three

What to praise (write inline):

- A student who notices their A values are *less dramatic* than the demo and traces it to the segmentation choice they made
- A student who picks a "boring" reticulate fish, gets A 1 as predicted, and reasons about why their lineage favors disruptive coloration
- Anyone who refers back to a lecture concept (Lec 11–12 material on cell palette + pattern mechanism) when interpreting their fish

---

Part 6 — Pre-lab and day-of checklist

Two days before lab

- ☐ Confirm Binder is warm — visit `https://mybinder.org/v2/gh/AlfaroLab/eeb187-pavo-lab/HEAD?urlpath=rstudio` and let it build. Subsequent students hitting the same commit get cached builds.
- ☐ Re-render `lab-05-pavo-intro.qmd` and `pre-lab-installation.qmd` if anything changed. Confirm the PDFs exist.
- ☐ Spot-check one student's installation per launch path (Native, Docker, Binder, Codespaces) is achievable end-to-end.

Day before lab

- ☐ Walk through `scripts/walkthrough.R` top-to-bottom yourself in a fresh RStudio session. Time how long it takes — should be ~25 minutes at a steady pace.
- ☐ Have the three pre-rendered metric tables (sergeant major / cleaner wrasse / mandarin) memorized so you can reveal them on demand.
- ☐ Print or have open this guide's Part 4 (pedagogical playbook).

Morning of lab

- ☐ Restart your laptop's RStudio. Run `library(pavo); library(magick); library(tidyverse); source("scripts/helpers.R")` to confirm everything still loads.
- ☐ Test `segment_fish()` on one of the demo fish to confirm rembg + magick are still on PATH. If `uvx` was last used > 1 month ago, the model cache may have been GC'd; running it once now downloads it.
- ☐ Take 5 student-fish images (3 you'll demo + 2 you'll reserve for "what if" student questions) and run them through the full pipeline as a smoke test.

In lab (90 min)

- See the time-budget table in §4.1.
- **Don't fall behind.** If demo Part A is ticking past 40 minutes, skip the m_dS/m_dL panel of Step 5 and let students read it in the worksheet.
- **Take notes on student questions** that surfaced. We update the troubleshooting boxes in this guide based on what comes up in real labs.

Appendix A — Glossary of metrics

For when a student asks "what does X mean?" in the middle of lab and you want a single sentence:

- **k** — number of color classes the image was reduced to.
- **m** — fraction of grid cells that are at a color boundary, i.e., adjacent to a cell of a *different* class. High m = busy pattern. Low m = uniform pattern.
- **A** — vertical-edge frequency divided by horizontal-edge frequency. **A > 1 = bars; A < 1 = stripes; A 1 = isotropic.**
- **Jc** — Simpson diversity of *color-class areas*. High Jc = many colors, evenly distributed. Low Jc = one or two colors dominate.
- **Jt** — Simpson diversity of *transitions*. High Jt = many different boundary types. Low Jt = pattern is dominated by one or two boundary types (e.g., black-yellow over and over).
- **m_dS** — average chromatic (saturation/hue) distance across edges. High m_dS = saturated, contrasty edges; low m_dS = subtle ones.
- **m_dL** — same as m_dS but for luminance (brightness). High m_dL = bright/dark contrast at edges.

Appendix B — Pre-loaded student-fish library

Students assigned to each lineage have these 5 species pre-loaded in the repo at `images/student-fish/<lineage>/`:

Lineage	Species
Chaetodontidae (butterflyfish)	*Chaetodon auriga*, *C. lunulatus*, *Forcipiger flavissimus*, *Hemitaurichthys polylepis*, *Heniochus acuminatus*
Pomacanthidae (angelfish)	*Centropyge bicolor*, *Holacanthus ciliaris*, *Pomacanthus imperator*, *P. paru*, *Pygoplites diacanthus*
Balistidae (triggerfish)	*Balistoides conspicillum*, *Melichthys indicus*, *Odonus niger*, *Pseudobalistes fuscus*, *Rhinecanthus aculeatus*
Acanthuridae (surgeonfish)	*Acanthurus lineatus*, *Naso lituratus*, *Paracanthurus hepatus*, *Zebrasoma flavescens*, *Z. xanthurum*

Students can use any three of their lineage's pre-loaded species without uploading anything. Or they can bring their own JPG and add it alongside the pre-loaded ones (see worksheet Part B Step 1 for image-folder instructions per OS).

Questions, errors, or improvements: post in the EEB 187 channel or PR against this file.